CPC — Lecture 10

Little Girl and Maximum Sum · Number of Ways · Fenwick tree (start)

Competitive Programming and Contests — Monday 19/10/2026

Sources. Recording `Lessons/L10/L_10.MP4` (85 min) and its automatic transcript `L10_transcript.txt` (not revised); the prof's handwritten board (frames of the recording, `board/l10_*.jpg`); the prof's 2025 notes `Lessons/L10/Prefix_sums_2025.pdf` (static prefix sums, Ilya, Little Girl, Number of Ways); the prof's blog posts *The Power of Prefix Sums* (`https://pages.di.unipi.it/rossano/blog/2023/prefixsums`) and *Dynamic Prefix Sums with Fenwick Tree* (`https://pages.di.unipi.it/rossano/blog/2023/fenwick`), code copied verbatim; your notes `Lessons/L10/notes.md`. The recording stops at 84:54, in the middle of the Fenwick tree (the iPad was running out of battery): the last part of §3 comes **only from your notes** and is marked as such. Everything marked [extra] was *not* said or written in the lecture (my explanations, examples, code).

Code: `Lessons/L10/src` (`cargo test -p l10`). Exercises: `Lessons/L10/practice`, E43–E46 (`cargo test -p practice_l10`).

Highlights: general / to remember good: works, efficient, correct bad: pitfalls, wrong ideas, costs definitions and key results

Where we are. L9 (14/10) ended Frogs and Mosquitoes and solved the first static prefix-sum problem, *Ilya and Queries* ("problem #1"). Today: **problem #2** *Little Girl and Queries* and **problem #3** *Number of Ways*, "the last problem about static prefix sums", then the start of **dynamic prefix sums** with the **Fenwick tree**. (This explains the numbering in your notes: the "problem #2" you could not find is Little Girl.)

Contents

1 Problem #2: Little Girl and Queries (Codeforces 276C) 2
- 1.1 The problem . . . 2
- 1.2 The idea (found in class) . . . 2
- 1.3 Step #1, solution 1: sweep line — $\Theta(n + m \log m)$ . . . 3
- 1.4 Step #1, solution 2: an array whose prefix sums are F — $\Theta(n + m)$ . . . 3

2 Problem #3: Number of Ways (Codeforces 466C) 5
- 2.1 Cubic and quadratic . . . 5
- 2.2 Linear . . . 5

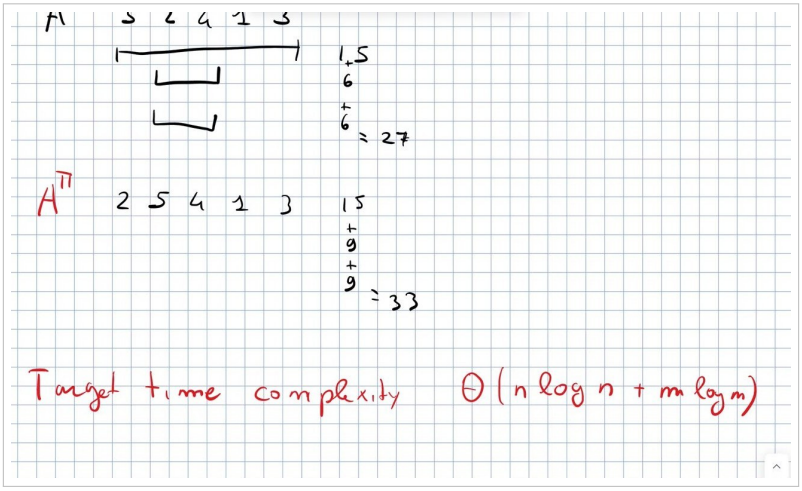
3 Dynamic prefix sums: the Fenwick tree 7
- 3.1 Why it matters . . . 7
- 3.2 The problem and the trivial solutions . . . 7
- 3.3 Relaxed version: Sum only at powers of two . . . 8
- 3.4 After the recording: from your notes . . . 8

4 Next time 10

1 Problem #2: Little Girl and Queries (Codeforces 276C)

1.1 The problem

Problem. Given an array $A[1, n]$ and a set Q of m queries, each query a RangeSum(i, j). Permute the values of A to maximize the sum of the results of the queries in Q ("we are free to permute the elements in any way we want; the elements must remain the same").
Example. $A = [5, 2, 4, 1, 3]$, queries $(1, 5)$, $(2, 3)$, $(2, 3)$: value $15 + 6 + 6 = 27$ (the prof says "26" at 03:56, a slip: the board says 27). With $A^{\pi} = [2, 5, 4, 1, 3]$: $15 + 9 + 9 =$ **33**, which is the optimum.



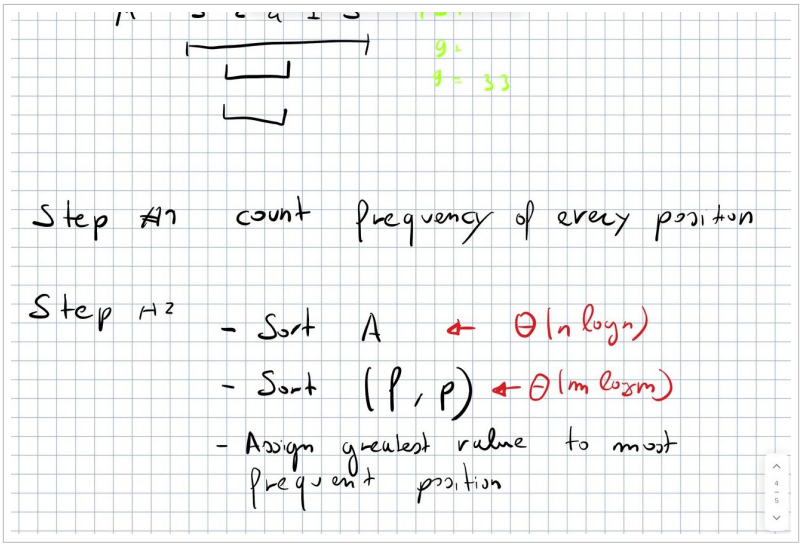
Board, 10:00 (recording L_10.MP4)

Target time complexity: $\Theta(n \log n + m \log m)$ (board; "can be improved a bit, we will discuss later"). [extra] Your notes write the two short queries as $(1, 2)$: on the board they are under the values $2, 4$, i.e. positions $2, 3$.

1.2 The idea (found in class)

For every position count how many queries touch it (its *frequency*); put the largest values in the most frequent positions. In the example the frequencies are $1, 3, 3, 1, 1$: 5 and 4 go to positions $2, 3$, then $1, 2, 3$ anywhere $\Rightarrow 15 + 9 + 9 = 33$.

- **Step #1**: count the frequency of every position.
- **Step #2**: sort A in decreasing order, $\Theta(n \log n)$; sort the pairs (frequency, position) in decreasing order, $\Theta(m \log m)$ on the board [extra] (there are n pairs, so strictly $\Theta(n \log n)$); assign the greatest value to the most frequent position.



Board, 16:00 (recording L_10.MP4)

Remark: with constraints on the values of A, or on m (frequencies are $\leq m$), counting sort removes the logarithmic factors.

1.3 Step #1, solution 1: sweep line — $\Theta(n + m \log m)$

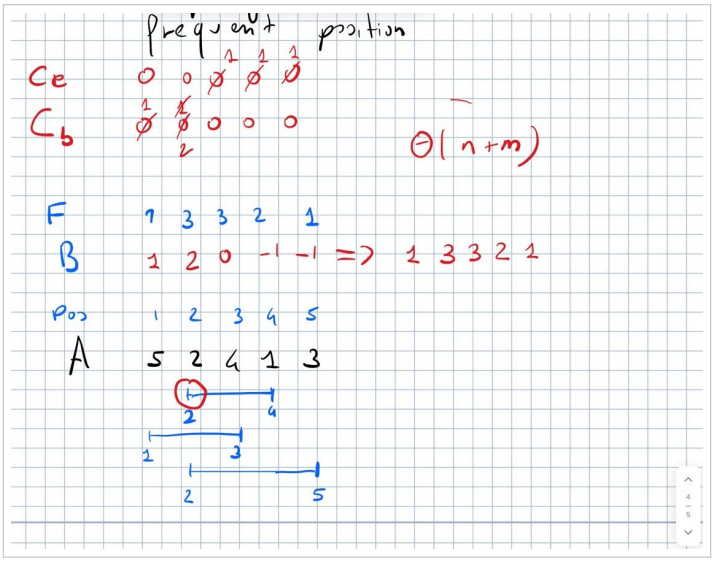
Proposed by a student, "something we already discovered" in the *maximum number of overlapping intervals* problem (L7): sort the begin/end points of the segments, move the sweep line, $+1$ at every beginning, -1 at every end; here we can also scan all the n positions to write down every frequency. Cost: $\Theta(m \log m)$ for sorting the $2m$ endpoints. (The prof did not write this one on the iPad.)

1.4 Step #1, solution 2: an array whose prefix sums are F — $\Theta(n + m)$

"Very useful because we are going to reuse the idea later." Queries $(2, 4)$, $(1, 3)$, $(2, 5)$ on $n = 5$: the target is $F = [1, 3, 3, 2, 1]$. Build an array B such that its prefix sums are F.

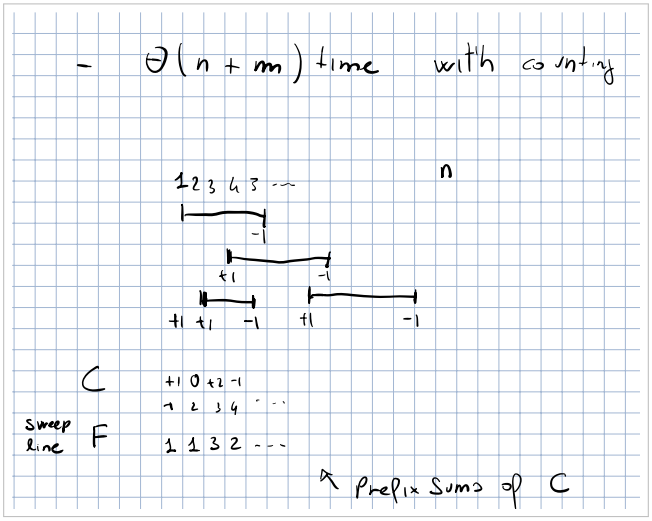
- A $+1$ in $B[i]$ is propagated by the prefix sums *until the end* of the array: $+1$ at 2 gives $0, 1, 1, 1, 1$.
- The segment $(2, 4)$ should give $0, 1, 1, 1, 0$: we must stop the propagation at the position immediately after the end of the segment, with a -1.
- So every query (i, j) does $B[i] \mathrel{+}= 1$ and $B[j + 1] \mathrel{-}= 1$ (nothing if $j + 1 > n$). Then $F =$ prefix sums of B.

Example: $B = [1, 2, 0, -1, -1] \Rightarrow F = [1, 3, 3, 2, 1]$. (The prof made a mistake on the first try, putting a -1 at the end instead of after it, and redid it from scratch: the -1 goes at $j + 1$, not at j.) Time $\Theta(n + m)$: zero out B, two updates per query, one prefix-sum pass. Better than the sweep line, which sorts.



Board, 36:20 (recording `L_10.MP4`)

But the sweep line can be linear too: counting sort. Do we really need a comparison-based sort $(m \log m)$ for the endpoints? No: the largest possible endpoint is n, so counting sort with an array of n counters works. We must keep the *satellite information* (beginning or end?): either $2n$ counters, or two arrays C_b (beginnings) and C_e (ends). In the example $C_b = [1, 2, 0, 0, 0]$, $C_e = [0, 0, 1, 1, 1]$ (positions of the ends). B is just a more direct representation of C_b and C_e: $+1/-1$ instead of "beginning/end". And the (stable) counting sort is itself doing prefix sums, to find where the elements equal to a key end in the sorted array: "we are rediscovering the same approaches often".



Prof.'s notes of **last year** (2025), `Prefix_sums_2025.pdf` p. 5

Lesson ("I would like to remark this"). Always consider the magnitude of the values in your task. If the numbers are bounded, direct-access tables instead of hashing, counting sort / radix sort instead of merge sort: much faster.

Prof's code [blog] (verbatim; `l10::little_girl`):

```rust
// We assumes queries are 0-based indexed
pub fn little_girl(a: &[i64], q: &[(usize, usize)]) -> i64 {
    if a.is_empty() {
        return 0;
    }

    let mut u = vec![0i64; a.len()];

    for &(l, r) in q {
        assert!(l <= r);
        assert!(r < u.len());

        u[l] += 1;
        if r + 1 < u.len() {
            u[r + 1] -= 1;
        }
    }

    let mut f = u
        .iter()
        .scan(0, |sum, e| {
            *sum += e;
            Some(*sum)
        })
        .collect::<Vec<_>>();

    // we sort both f and a in decreasing order, nothing changes
    f.sort_unstable();
    let mut a_sorted = a.to_vec();
    a_sorted.sort_unstable();

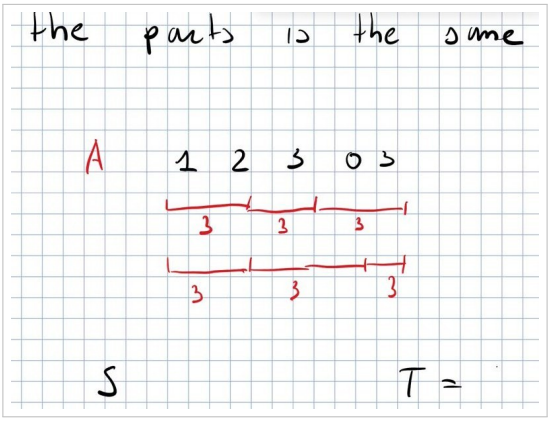
    a_sorted
```

```
        .iter()
        .zip(f)
        .fold(0, |result, (value, freq)| result + value * freq)
}
```

[extra] `u` is the array B. Both arrays are sorted in *increasing* order: pairing the k-th smallest with the k-th smallest is the same as pairing the largest with the largest. Total: $\Theta(m + n \log n)$.

2 Problem #3: Number of Ways (Codeforces 466C)

"The last problem about *static* prefix sums" (static: the array does not change after computing the prefix sums).
Problem. Given $A[1, n]$, count in how many ways we can split A into 3 parts so that the sums of the parts are the same. To make it interesting, the values can be zero or negative: with positive values only there is at most one split (just check 0 or 1).
Example. $A = [1, 2, 3, 0, 3]$: 2 ways, $[1, 2 \mid 3 \mid 0, 3]$ and $[1, 2 \mid 3, 0 \mid 3]$, all parts sum to 3.



Board, 42:00 (recording `L_10.MP4`)

Target: linear time.

2.1 Cubic and quadratic

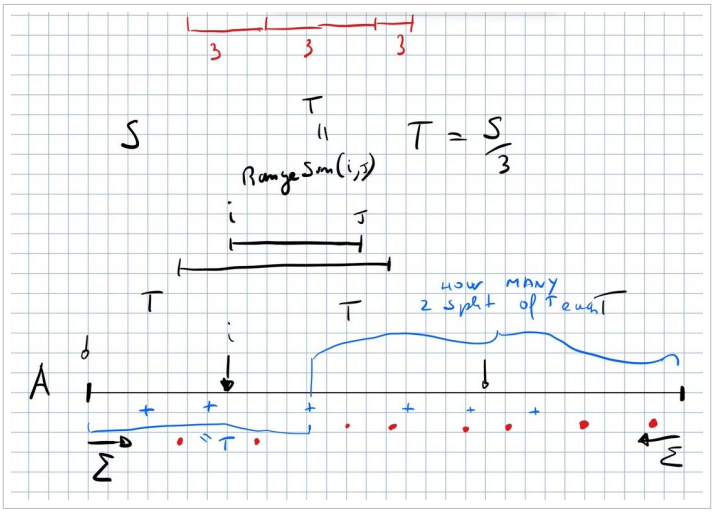
Cubic: try all the splittings and compute the sums. Quadratic: S = sum of A, target $T = S/3$; with prefix sums, fix the central part $[i, j]$ (quadratically many) and check $\text{RangeSum}(i, j) = T$ in $O(1)$. If it is, the prefix and the suffix sum to $2T$ together: check that the prefix is T (then the suffix is T too, the total being $3T$).

2.2 Linear

The class went towards two pointers, then towards counting. Summary given by the prof:

1. Left to right, compute the prefix sums and mark with "+" (blue) every position where the prefix sum is T: each one is the end of a valid first part. "We may have many of them" (zeros and negative numbers).
2. For each "+" we want how many splits of the remaining suffix into 2 parts of sum T each, and sum these numbers. It is not enough to find one j, and scanning the remaining part for every "+" is quadratic.
3. Right to left, compute suffix sums and mark with a (red) dot every position where the suffix sum is T: the beginning of a valid third part. Then the middle part sums to $3T - T - T = T$ automatically.
4. Answer = $\sum$ over the "+" of the number of red dots to its right. Not "#blue $\times$ #red": that counts pairs with the red dot on the left (double counting).

5. "How many red points on the right?" is a suffix sum on 0/1 indicators (1 if the position is red): precomputed, $\Theta(n)$ overall.



Board, 59:40 (recording `L_10.MP4`)

[extra] "To the right" must leave a non-empty middle part: if the first part ends at i, the third part starts at $\geq i+2$. Last year the prof also warmed up with the easier problem of splitting into 2 parts (one scan, check $p = S/2$): not done this year, see p. 6 of the 2025 notes.

Prof's code [blog] (verbatim; `l10::number_of_ways`):

```rust
pub fn number_of_ways(a: &[i64]) -> usize {
    let sum: i64 = a.iter().sum();

    if sum % 3 != 0 {
        return 0;
    }

    let target = sum / 3;
    let mut c: Vec<_> = a
        .iter()
        .rev()
        .scan(0, |sum, e| {
            *sum += e;
            Some(*sum)
        })
        .scan(0, |counter, sum| {
            if sum == target {
                *counter += 1usize
            };
            Some(*counter)
        })
        .collect();

    c.reverse();

    let mut result = 0;
    let mut sum = 0;
    for (i, &v) in a[..a.len() - 2].iter().enumerate() {
        sum += v;
        if sum == target {
```

```
            result += c[i + 2];
        }
    }

    result
}
```

[extra] $c[i]$ = number of red dots at positions $\geq i$ (the suffix sum on the indicators); `c[i + 2]` leaves at least one element in the middle. Pitfall: with $n = 1$ (allowed by Codeforces) `a.len() - 2` underflows and the function panics (`number_of_ways(&[0])`); guard with `if a.len() < 3 { return 0; }`. Tested in `l10` against an $O(n^2)$ brute force.

3 Dynamic prefix sums: the Fenwick tree

3.1 Why it matters

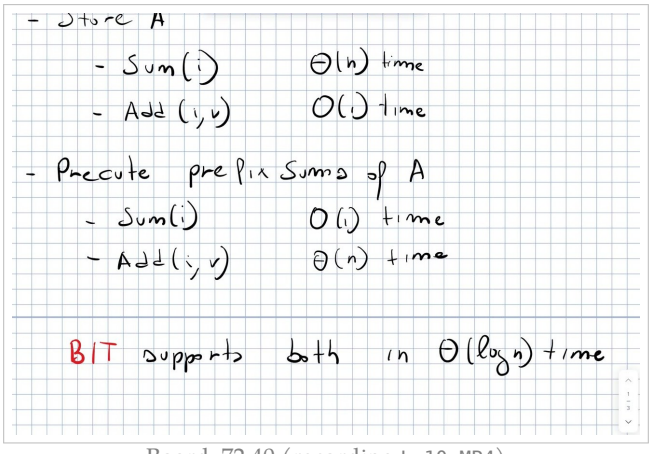
"One of the most elegant data structures we are going to see in this course."

- Both operations in $O(\log n)$, "just three instructions per operation", very fast.
- No more space than the array it indexes: implicit — a binary tree stored in an array, the structure is in the order of the elements, and we move between parent and children with binary operations on the index (hence the other name, **Binary Indexed Tree, BIT**).
- Alternative: the **segment tree** (coming next): same time, more space, but easy to modify for other operations (minimum of a subarray, ...). The Fenwick tree is mostly used for dynamic prefix sums.
- Named after its inventor, P. Fenwick (New Zealand), who used it for *arithmetic coding*: while scanning a text you keep the frequencies of the symbols updated and need how many occurrences of symbols smaller than b there are, i.e. prefix sums on dynamic frequencies.

3.2 The problem and the trivial solutions

Given $A[1, n]$, support $\mathsf{Sum}(i) = \sum_{k=1}^{i} A[k]$ and $\mathsf{Add}(i, v)$: $A[i] \mathrel{+}= v$ (v may be negative). $\mathsf{RangeSum}(i, j) = \mathsf{Sum}(j) - \mathsf{Sum}(i-1)$, so we only discuss Sum ("easier to implement").

	$\mathsf{Sum}(i)$	$\mathsf{Add}(i, v)$
store A	$\Theta(n)$	$O(1)$
store the prefix sums of A	$O(1)$	$\Theta(n)$ (all following prefix sums change)
BIT	$\Theta(\log n)$	$\Theta(\log n)$



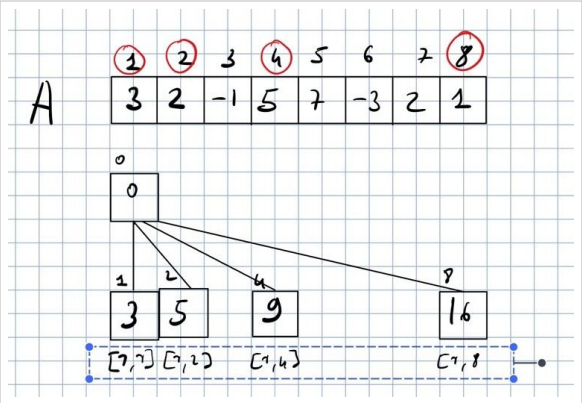
Board, 72:40 (recording `L_10.MP4`)

The trivial solutions are useful only with a huge imbalance: one operation done a linear number of times and the other asymptotically fewer than $\log n$ times (or, a student noted, a group of all the adds followed by all the sums). In any other case, the vast majority, the BIT is better.

3.3 Relaxed version: Sum only at powers of two

Running example ("for a while"): $A = [3, 2, -1, 5, 7, -3, 2, 1]$, positions $1..8$.

- **Relaxed problem**: Sum(i) only if i is a power of 2 (undefined otherwise); Add(i, v) for every i.
- Checklist entry for data-structure problems: how many possible questions are there? If few, precompute the answers. Here only $\log n$ positions can be asked: store Sum$(1) = 3$, Sum$(2) = 5$, Sum$(4) = 9$, Sum$(8) = 16$.
- Sum: $O(1)$. Add(i, v): update the stored answers at the powers of two $\geq i$, $O(\log n)$ (worst case $i = 1$, all of them change). Example: Add$(6, 5)$ changes only position 8: $16 \to 21$.
- "I promised a tree": add a fake root (position 0, value 0) and connect it to the stored nodes. Write the range of each node, $[1, 1], [1, 2], [1, 4], [1, 8]$, and the positions in binary: $0001, 0010, 0100, 1000$. "What's the characteristic of these positions?" [extra] exactly one bit set to 1.



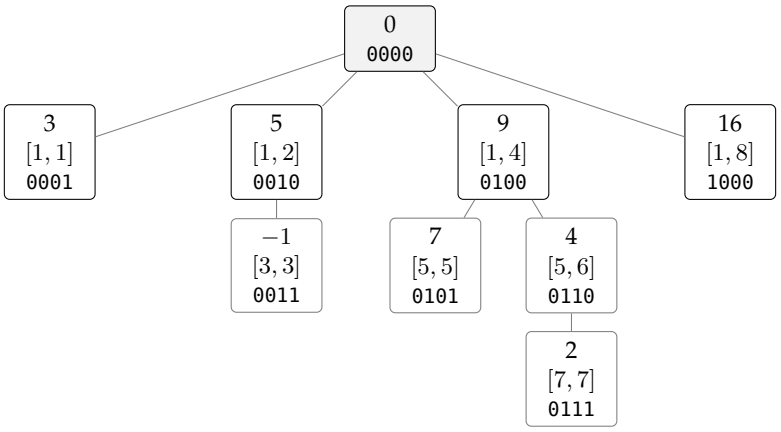
Board, 84:00 (recording `L_10.MP4`)

3.4 After the recording: from your notes

The recording ends here (84:54). What follows is from your notes only (`notes.md`); figure and [extra] parts are mine. To be checked against L11 if the prof recaps.

- Cover the uncovered parts by applying the same principle recursively. Between 4 and 8 the uncovered part is $[7, -3, 2]$ (positions $5, 6, 7$, i.e. $1, 2, 3$ shifted by 4): its relaxed solution stores $[5, 5] = 7$ and $[5, 6] = 4$, positions 0101 and 0110 (two bits set), linked to their parent, node 4. Then $[-1]$ at position $3 = 0011$, range $[3, 3]$, child of node 2; and [extra] 2 at position $7 = 0111$, range $[7, 7]$, child of node 6.
- Number of trailing 0s in the binary representation of a node = number of its children.
- Sum(i): start from the node whose range ends at i (node i) and sum its value and the values of its ancestors up to the root.
- Add(i, v): update the values of all the nodes whose range contains i. (Running times: not written in your notes.)

[extra] The whole tree of the example (value, range, position in binary):



[extra] Node i covers $(i - \text{lsb}(i), i]$, where $\text{lsb}(i)$ is the lowest set bit of i. The parent of i is $i - \text{lsb}(i)$ (clear the lowest set bit), so Sum visits at most $\log n + 1$ nodes: $\text{Sum}(7) = T[7] + T[6] + T[4] = 2 + 4 + 9 = 15$. Add visits $i,\ i + \text{lsb}(i), \ldots$ (the "next sibling" in the blog), again $O(\log n)$ nodes: $\text{Add}(3, v)$ touches $3, 4, 8$. Stored as an array, the tree is $[0 \mid 3, 5, -1, 9, 7, 4, 2, 16]$: $n + 1$ integers, no pointers.

Prof's code [blog] (verbatim; `l10::fenwick::FenwickTree`):

```rust
#[derive(Debug)]
pub struct FenwickTree {
    tree: Vec<i64>,
}

impl FenwickTree {
    pub fn with_len(n: usize) -> Self {
        Self {
            tree: vec![0; n + 1],
        }
    }

    pub fn len(&self) -> usize {
        self.tree.len() - 1
    }

    /// Indexing is 0-based, even if internally we use 1-based indexing
    pub fn add(&mut self, i: usize, delta: i64) {
        let mut i = i + 1;
        assert!(i < self.tree.len());

        while i < self.tree.len() {
            self.tree[i] += delta;
            i = Self::next_sibling(i);
        }
    }

    /// Indexing is 0-based, even if internally we use 1-based indexing
    pub fn sum(&self, i: usize) -> i64 {
        let mut i = i + 1;

        assert!(i < self.tree.len());
        let mut sum = 0;
        while i != 0 {
```

```
            sum += self.tree[i];
            i = Self::parent(i);
        }

        sum
    }

    pub fn range_sum(&self, l: usize, r: usize) -> i64 {
        self.sum(r) - if l == 0 { 0 } else { self.sum(l - 1) }
    }

    fn isolate_trailing_one(i: usize) -> usize {
        if i == 0 {
            0
        } else {
            1 << i.trailing_zeros()
        }
    }

    fn parent(i: usize) -> usize {
        i - Self::isolate_trailing_one(i)
    }

    fn next_sibling(i: usize) -> usize {
        i + Self::isolate_trailing_one(i)
    }
}
```

[extra] Tested in `l10::fenwick`: on the example (internal array $[0, 3, 5, -1, 9, 7, 4, 2, 16]$) and against the trivial array on random add/sum sequences.

4 Next time

- Continue the Fenwick tree (the lecture ended in the middle of it): why it works, running times, and applications.
- Announced in this lecture: the **segment tree** ("more space, but easy to modify to solve other problems"). The prof's 2025 notes on it are in `Lessons/L11/Segment_tree_2025.pdf` (dynamic prefix sums, RMQ, RMQ with occurrences, Search, #occurrences, successor, triplets, lazy propagation, persistent segment tree).
- No homework was assigned in this lecture.